

# Homework 1

## Homework 1

- Released today in Blackboard
- Due: 11:59pm, Mar. 18
- No late homework will be accepted!

▼ 算法设计与分析

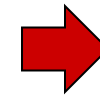
主页

公告

Slides

Homework 

Piazza

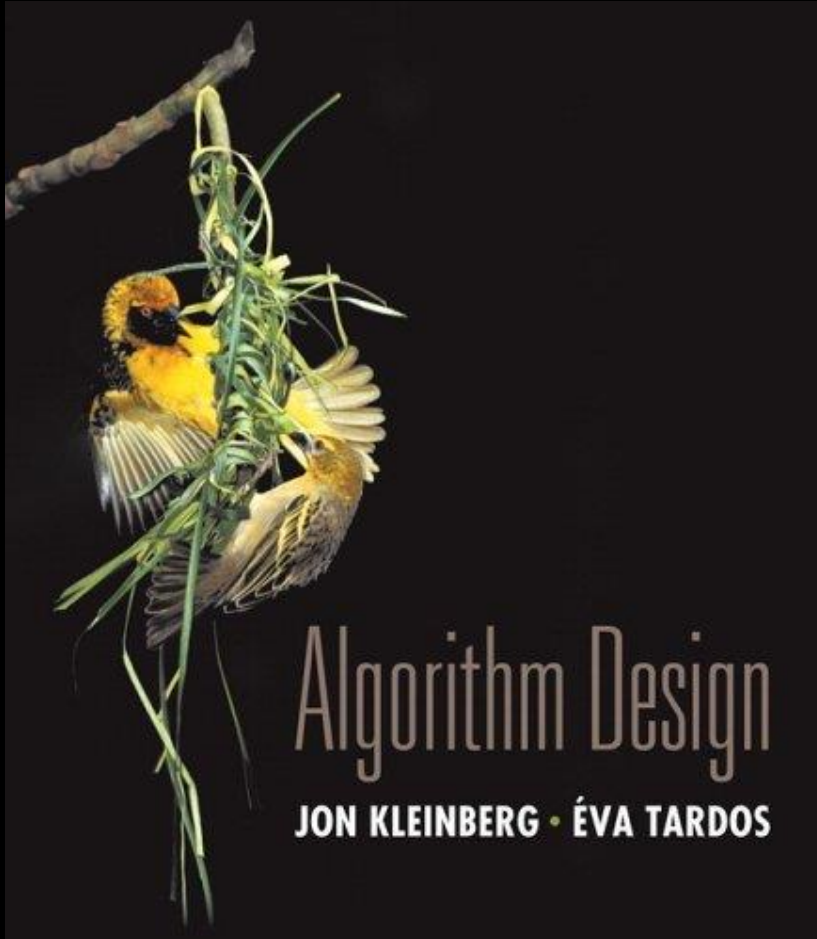


## Submission

- Register at gradescope.com using your university email
  - Make sure that your FULL NAME is your Chinese name and your STUDENT ID is correct
- Save your solutions in a PDF or image file
- Upload your file to Gradescope and match your solution to each problem

# Chapter 4

## Greedy Algorithms



Slides by Kevin Wayne.  
Copyright © 2005 Pearson-Addison Wesley.  
All rights reserved.

# Greedy Algorithms

A greedy algorithm is an algorithm that **makes the locally optimal choice** at each step.

## 4.7 Clustering

---

# Clustering

photos, documents, micro-organisms



**Clustering.** Given a set  $U$  of  $n$  objects labeled  $p_1, \dots, p_n$ , partition into clusters s.t. objects in different clusters are far apart.



e.g., a large number of corresponding pixels whose intensities differ over some threshold

**Fundamental problem.** Divide into clusters so that points in different clusters are far apart.

- Routing in mobile ad hoc networks.
- Identify patterns in gene expression.
- Document categorization for web search.
- Similarity searching in medical image databases
- Skycat: cluster  $10^9$  sky objects into stars, quasars, galaxies.

# Clustering of Maximum Spacing

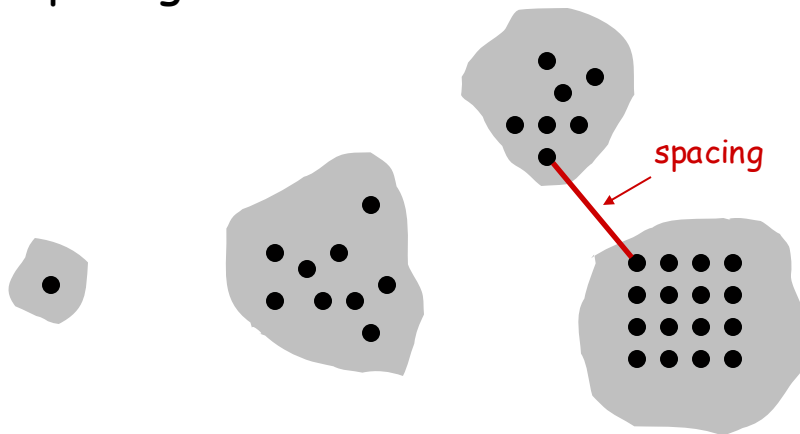
**k-clustering.** Divide objects into  $k$  non-empty groups.

**Distance function.** Assume it satisfies several natural properties.

- $d(p_i, p_j) = 0$  iff  $p_i = p_j$  (identity of indiscernibles)
- $d(p_i, p_j) \geq 0$  (nonnegativity)
- $d(p_i, p_j) = d(p_j, p_i)$  (symmetry)

**Spacing.** **Min** distance between any pair of points in different clusters.

**Clustering of maximum spacing.** Given an integer  $k$ , find a  $k$ -clustering of **maximum** spacing.



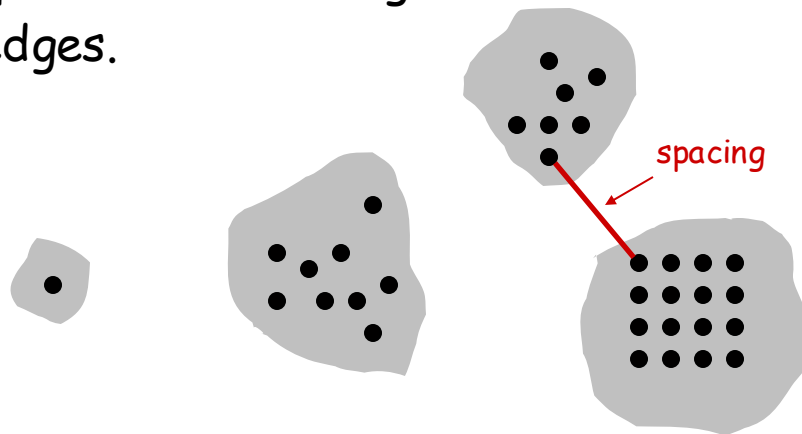
# Greedy Clustering Algorithm

## Single-link k-clustering algorithm.

- Create  $n$  clusters, one for each object.
- Find the closest pair of objects such that each object is in a different cluster; add an edge between them and merge the two clusters.
- Repeat  $n-k$  times until there are exactly  $k$  clusters.

**Key observation.** This procedure is precisely Kruskal's algorithm (except we stop when there are  $k$  connected components).

**Remark.** Equivalent to finding an MST and deleting the  $k-1$  most expensive edges.



# Greedy Clustering Algorithm: Analysis

**Theorem.** Let  $C^*$  denote the clustering  $C^*_1, \dots, C^*_k$  formed by deleting the  $k-1$  most expensive edges of an MST.  $C^*$  is a  $k$ -clustering of max spacing.

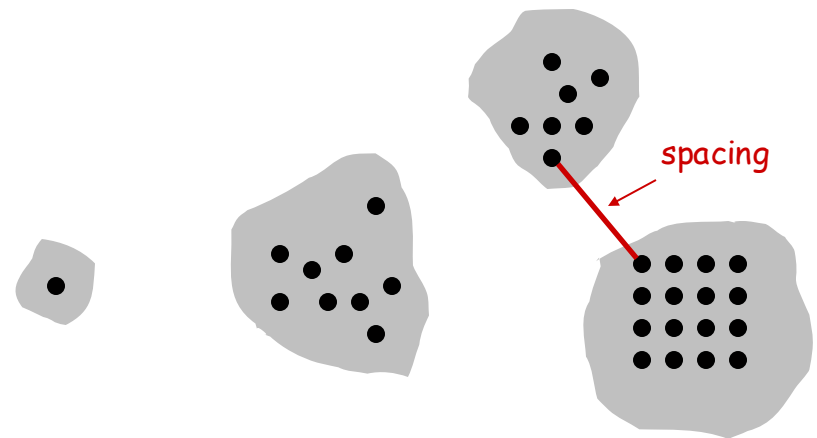
**Recall of Kruskal's algorithm.** [Kruskal, 1956]

Consider edges in ascending order of weight.

Case 1: If adding  $e$  to  $T$  creates a cycle,  
discard  $e$  according to cycle property.

Case 2: Otherwise, insert  $e = (u, v)$  into  $T$   
according to cut property where  $S$  = set of  
nodes in  $u$ 's connected component.

Until  $(n-1)$  edges have been collected.



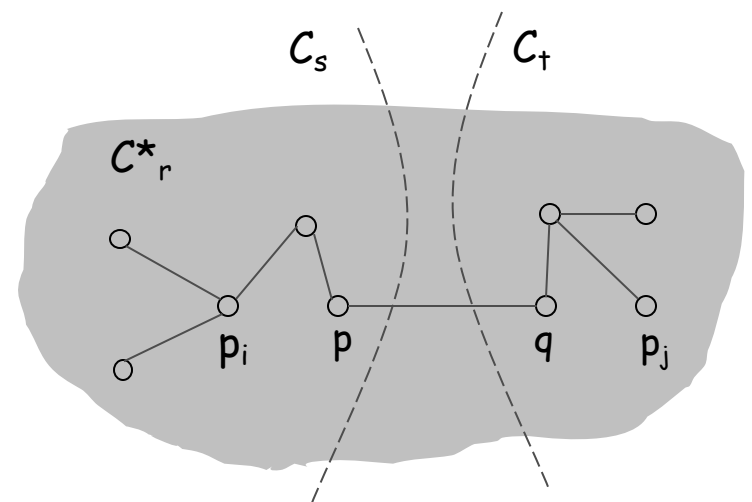


# Greedy Clustering Algorithm: Analysis

**Theorem.** Let  $C^*$  denote the clustering  $C^*_1, \dots, C^*_k$  formed by deleting the  $k-1$  most expensive edges of an MST.  $C^*$  is a  $k$ -clustering of max spacing.

**Pf.** Let  $C$  denote some other clustering  $C_1, \dots, C_k$ .

- The spacing of  $C^*$  is the length  $d^*$  of the  $(k-1)^{\text{st}}$  most expensive edge in MST.
- Let  $p_i, p_j$  be in the same cluster in  $C^*$ , say  $C^*_r$ , but different clusters in  $C$ , say  $C_s$  and  $C_t$ .
- Some edge  $(p, q)$  on  $p_i$ - $p_j$  path in  $C^*_r$  spans two different clusters in  $C$ .
- All edges on  $p_i$ - $p_j$  path have length  $\leq d^*$  since Kruskal chose them.
- Spacing of  $C$  is  $\leq d^*$  since  $p$  and  $q$  are in different clusters. ▀



## 4.3 Optimal Caching

---

# Optimal Offline Caching

## Caching.

- Cache with capacity to store  $k$  items.
- Sequence of  $m$  item requests  $d_1, d_2, \dots, d_m$ .
- Cache hit: item already in cache when requested.
- Cache miss: item not already in cache when requested: must bring requested item into cache, and evict some existing item, if full.

**Goal.** Eviction schedule that minimizes number of evictions.

**Ex:**  $k = 2$ , initial cache =  $ab$ ,  
requests:  $a, b, c, b, c, a, a, b$ .

**Optimal eviction schedule:** 2 evictions.

|          |       |   |
|----------|-------|---|
| a        | a     | b |
| b        | a     | b |
| c        | c     | b |
| b        | c     | b |
| c        | c     | b |
| a        | a     | b |
| a        | a     | b |
| b        | a     | b |
| requests | cache |   |

## Optimal Offline Caching: Farthest-In-Future

**Farthest-in-future.** Evict item in the cache that is not requested until farthest in the future.

current cache: 

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| a | b | c | d | e | f |
|---|---|---|---|---|---|

future queries:    g a b c e d a b b a c d e a f a d e f g h ...

↑  
cache miss

↑  
eject this one

**Theorem.** [Bellady, 1960s] FF is optimal eviction schedule.

**Pf.** Algorithm and theorem are intuitive; proof is subtle.

Will prove it later.

# Reduced Eviction Schedules

**Def.** A **reduced** schedule is a schedule that only inserts an item into the cache in a step in which that item is requested.

**Intuition.** Can transform an unreduced schedule into a reduced one with no more evictions.

|   |   |   |   |
|---|---|---|---|
| a | a | b | c |
| a | a | x | c |
| c | a | d | c |
| d | a | d | b |
| a | a | c | b |
| b | a | x | b |
| c | a | c | b |
| a | a | b | c |
| a | a | b | c |

an unreduced schedule

|   |   |   |   |
|---|---|---|---|
| a | a | b | c |
| a | a | b | c |
| c | a | b | c |
| d | a | d | c |
| a | a | d | c |
| b | a | d | b |
| c | a | c | b |
| a | a | c | b |
| a | a | c | b |

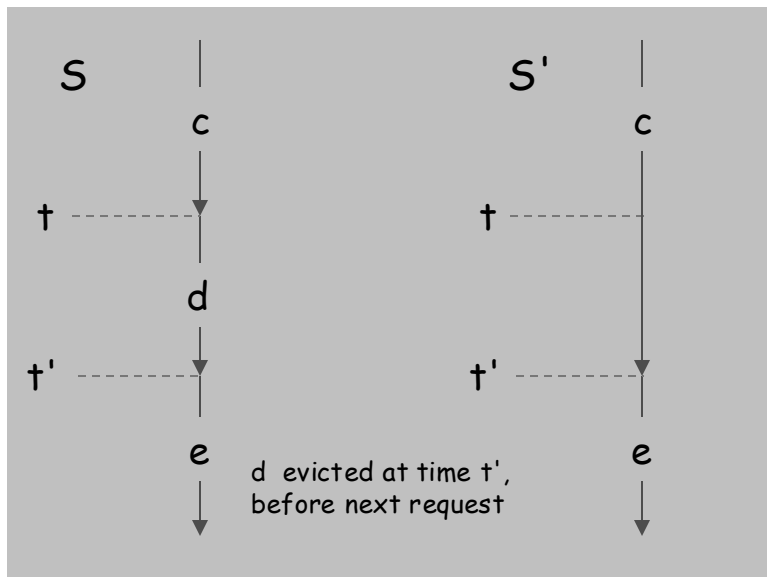
a reduced schedule

# Reduced Eviction Schedules

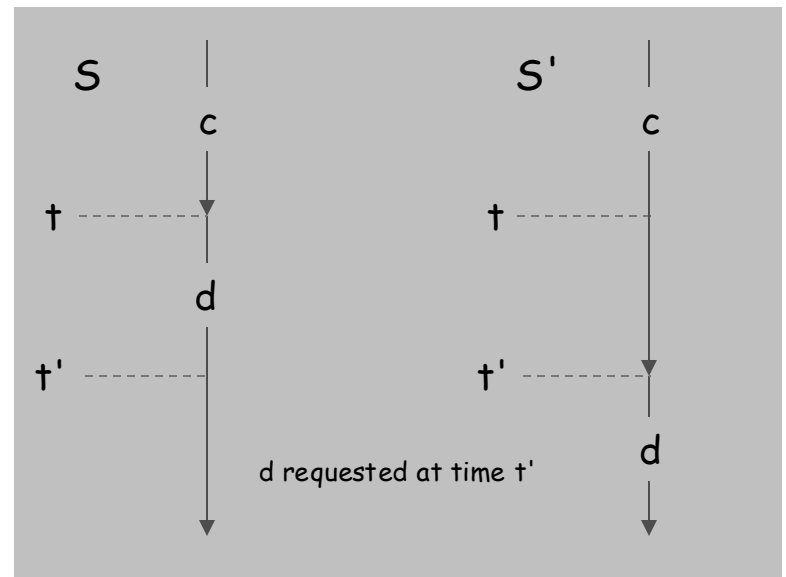
**Claim.** Given any unreduced schedule  $S$ , can transform it into a reduced schedule  $S'$  with no more evictions.

**Pf.** (by induction on number of unreduced items) ← doesn't enter cache at requested time

- Suppose  $S$  brings  $d$  into the cache at time  $t$ , without a request.
- Let  $c$  be the item  $S$  evicts when it brings  $d$  into the cache.
- Case 1:  $d$  evicted at time  $t'$ , before next request for  $d$ .
- Case 2:  $d$  requested at time  $t'$  before  $d$  is evicted. ▀



Case 1



Case 2

## Farthest-In-Future: Analysis

**Lemma.** Let  $S$  be a reduced schedule that makes the same schedule as  $S_{FF}$  through the first  $j$  requests. Then there is a reduced schedule  $S'$  that makes the **same** schedule as  $S_{FF}$  through the first  $j+1$  requests, and incurs no more evictions than  $S$  does.

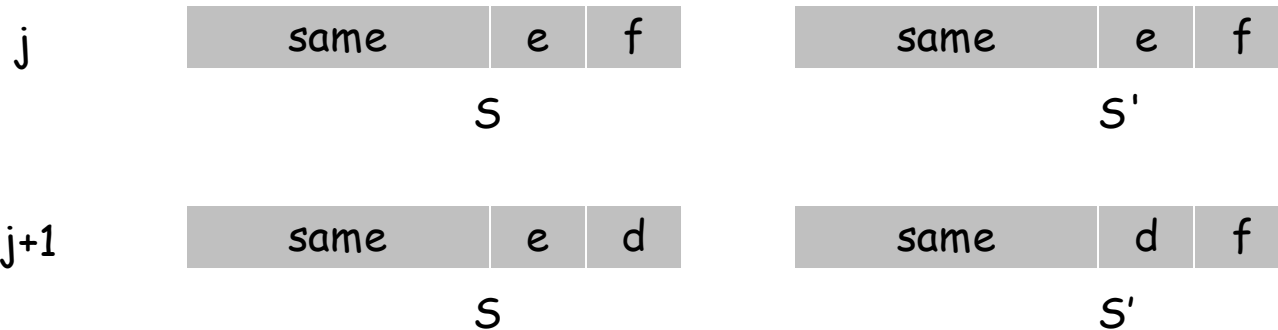
**Pf.**

- Consider  $(j+1)^{st}$  request  $d = d_{j+1}$ .
- Since  $S$  and  $S_{FF}$  have agreed up until now, they have the same cache contents before request  $j+1$ .
- Case 1: ( $d$  is already in the cache). Let  $S' = S$ .
- Case 2: ( $d$  is not in the cache and  $S$  and  $S_{FF}$  evict the same element).  
Let  $S' = S$ .

# Farthest-In-Future: Analysis

Pf. (continued)

- Case 3: ( $d$  is not in the cache;  $S_{FF}$  evicts  $e$ ;  $S$  evicts  $f \neq e$ ).
  - begin construction of  $S'$  from  $S$  by evicting  $e$  instead of  $f$



- now  $S'$  agrees with  $S_{FF}$  on first  $j+1$  requests
- From request  $j+2$  onward, we make  $S'$  the same as  $S$ , but this becomes impossible when  $e$  or  $f$  is involved

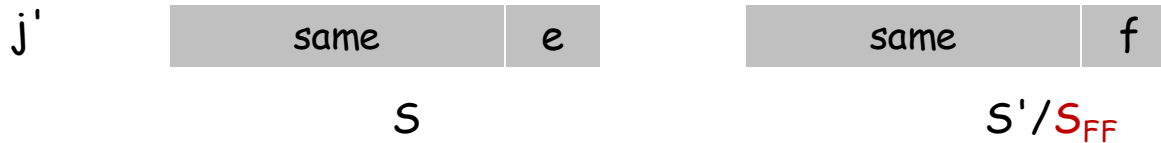


# Farthest-In-Future: Analysis

Pf. (continued)

Let  $j'$  be the **first** time after  $j+1$  that  $S$  and  $S'$  take a different action, and let  $g$  be item requested at time  $j'$ .

↑  
must involve  $e$  or  $f$  (or both)

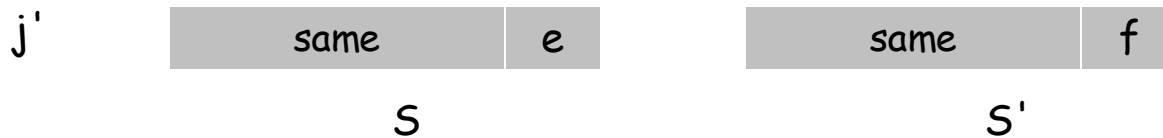


- Case 3a:  $g = e$ . Can't happen with Farthest-In-Future since there must be a request for  $f$  before  $e$ .

# Farthest-In-Future: Analysis

Pf. (continued)

Let  $j'$  be the **first** time after  $j+1$  that  $S$  and  $S'$  take a different action, and let  $g$  be item requested at time  $j'$ .

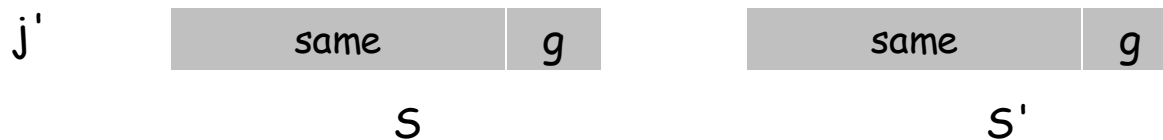


otherwise  $S'$  would take the same action



□ Case 3b:  $g \neq e, f$ .  $S$  must evict  $e$ .

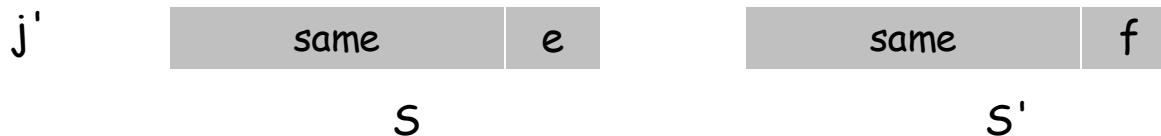
Make  $S'$  evict  $f$ ; now  $S$  and  $S'$  have the same cache. ▀



# Farthest-In-Future: Analysis

Pf. (continued)

Let  $j'$  be the **first** time after  $j+1$  that  $S$  and  $S'$  take a different action, and let  $g$  be item requested at time  $j'$ .



- Case 3c:  $g = f$ . Element  $f$  can't be in cache of  $S$ , so let  $e'$  be the element that  $S$  evicts.
  - if  $e' = e$ ,  $S'$  accesses  $f$  from cache; now  $S$  and  $S'$  have same cache
  - if  $e' \neq e$ ,  $S'$  evicts  $e'$  and brings  $e$  into the cache; now  $S$  and  $S'$  have the same cache.  $S'$  is no longer reduced, but can be transformed into a reduced schedule which
    - agrees with  $S_{FF}$  through step  $j+1$
    - has no more evictions than  $S$

# Farthest-In-Future: Analysis

**Theorem.** FF is optimal eviction algorithm.

**Pf.** (by induction on number of requests  $j$ )

Base case (trivial):

There exists an optimal reduced schedule  $S$  that makes the same schedule as  $S_{FF}$  through the first  $0$  requests.

Inductive step (implied by the lemma):

If there exists an optimal reduced schedule  $S$  that agrees with  $S_{FF}$  through the first  $j$  requests,

then there exists an optimal reduced schedule  $S'$  that agrees with  $S_{FF}$  through the first  $j+1$  requests

# Caching Perspective

## Online vs. offline algorithms.

- Offline: full sequence of requests is known a priori.
- Online (reality): requests are not known in advance.
- Caching is among most fundamental online problems in CS.

**LIFO.** Evict page brought in most recently.

**LRU.** Evict page whose most recent access was earliest.

↑  
FF with direction of time reversed!

**Theorem.** FF is optimal offline eviction algorithm.

- Provides basis for understanding and analyzing online algorithms.
- LRU is  $k$ -competitive. [Section 13.8]
- LIFO is arbitrarily bad.

# Greedy Algorithms: Chapter Summary

---

# Greedy Algorithms

## Basic idea

- Make the locally optimal choice at each step.

## Algorithms

- Interval Scheduling
  - Choose the job with the earliest finish time
- Scheduling to Minimize Lateness
  - Choose the job with the earliest deadline
- Clustering
  - Single-link k-clustering
- Optimal Caching
  - Evict item that is requested farthest in future

# Greedy Algorithms

## Proof skills

- **Greedy algorithm stays ahead.** Show that after each step of the greedy algorithm, its solution is at least as good as any other algorithm's.
- **Exchange argument.** Gradually transform any solution to the one found by the greedy algorithm without hurting its quality.